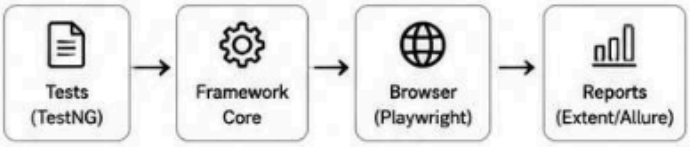
PLAYWRIGHT AUTOMATION FRAMEWORK

Playwright

WHAT IS AN AUTOMATION FRAMEWORK?

An Automation Framework is a set of guidelines, best practices, and reusable components that help us build scalable, maintainable and reliable test automation solutions.

HIGH LEVEL ARCHITECTURE



PROJECT STRUCTURE

```
Playwright-Framework/
├── src/
│   ├── base/
│   ├── factory/
│   ├── pages/
│   ├── utils/
│   ├── listeners/
│   ├── constants/
│   ├── config/
│   ├── resources/
│   └── reports/
├── tests/
├── test-data/
├── playwright.config.ts
├── testng.xml
├── pom.xml
├── package.json
└── README.md
```

KEY BENEFITS

- Scalable & Maintainable
- Reusable Components
- Easy to Integrate with CI/CD
- Parallel Execution Support
- Better Reporting & Logging
- Environment & Data Management

DESCRIPTION OF EACH COMPONENT

- **src/** — Contains all the source code of the framework.
- **base/** — Holds base classes like BaseTest, BasePage which contain common setup, teardown and reusable methods.
- **factory/** — Contains DriverFactory or BrowserFactory to handle browser and context initialization.
- **pages/** — Page Object classes. Each page of the application will have a separate class with locators and actions.
- **utils/** — Utility classes with reusable methods like WaitUtils, DateUtils, RandomUtils, FileUtils, etc.
- **listeners/** — TestNG Listeners for reporting, logs, screenshots on failure, retry analyzer, etc.
- **constants/** — Holds constant values like timeouts, messages, file paths, keys, etc.
- **config/** — Configuration related classes to read properties from config files (env, urls, credentials, etc).
- **resources/** — Contains external resources like properties files, yaml/json files, test data, templates, etc.
- **reports/** — Generated reports will be stored here (Extent, Allure, Playwright HTML Report, screensions).

tests/	Contains all the test classes written using TestNG.
test-data/	Stores test data files like excel, csv, json, yaml, etc.
playwright.config.ts	Playwright configuration file. Defines browsers, timeouts, retries, reporters, projects, etc.
testng.xml	TestNG suite file. Used to group and run test classes.
pom.xml	Maven project object model. Manages dependencies and build configurations.
package.json	Node.js dependencies and scripts (e.g., playwright install).
README.md	Project documentation and setup instructions.

FRAMEWORK HIGHLIGHTS

Page Object Model | Data Driven Testing | Parallel Execution | Cross Browser Support | Reporting & Screenshots | Logging & Monitoring

PLAYWRIGHT AUTOMATION FRAMEWORK

CORE CLASSES OF THE FRAMEWORK

These are the backbone classes that handle setup, browser initialization and common reusable functionality.

1 BaseTest

Manages test lifecycle. Handles setup, teardown, screenshots and browser management.

BaseTest.ts

```ts
import { test as base, Browser, Page } from '@playwright/test';
import { BrowserFactory } from '../factory/BrowserFactory';
import { ConfigReader } from '../config/ConfigReader';

export class BaseTest {
    protected static browser: Browser;
    protected page: Page;

    // Runs once before all tests
    static async globalSetup() {
      const browserType = ConfigReader.get('browser');
      browser = await BrowserFactory.getBrowser(browserType);
  }

  // Runs before each test
  async setup() {
     const context = await browser.newContext();
     this.page = await context.newPage();
     await page.setDefaultTimeout(30000);
  }

  // Runs after each test
  async teardown(testInfo: any) {
    if (testInfo.status !== testInfo.expectedStatus) {
        await this.page.screenshot({ path: './reports/screenshots/${testInfo-
        title-${Date.now()}.png', fullPage: true });
    }
    await this.page.close();
  }

  // Runs once after all tests
  static async globalTeardown() {
    if (browser) await browser.close();

}
```

2 BrowserFactory

Creates and returns browser instance based on configuration.

BrowserFactory.ts

```ts
import { chromium, firefox, webkit, Browser } from '@playwright/test';
import { ConfigReader } from '../config/ConfigReader';

export class BrowserFactory {
   static async getBrowser(browserName: string): Promise<Browser> {
     const isHeadless = ConfigReader.getBooleen('headless');
     const args = ConfigReader.getList('browserArgs');

     switch (browserName.toLowerCase()) {
       case 'chrome':  return await chromium.launch({ headless: isHeadless, args });
       case 'firefox': return await firefox.launch({ headless: isHeadless, args });
       case 'webkit':  return await webkit.launch({ headless: isHeadless, args });
       default: throw new Error('Browser ${browserName) is not supported');
     }
   }
}
```

3 BasePage

Base class for all pages. Contains common methods and web actions.

BasePage.ts

```ts
import { Page, Locator, expect } from '@playwright/test';

export class BasePage {
   constructor(protested page: Page) {}

   async click(locator: Locator) { await locator.click(); }
   async type(locator: Locator, text: string) { await locator.fill(text); }
   async getText(locator: Locator): Promise<string> {
      return await locator.textContment() || '';
   }
   async isVisible(locator: Locator): Promise<boolean> {
      return await locator.isVisible();
   }
   async waitForVisible(locator: Locator, timeout = 10000) {
      await locator.waitFor({ state: 'visible', timeout });
}
```

PLAYWRIGHT AUTOMATION FRAMEWORK

PAGE OBJECTS, CONFIG & UTILITIES

Reusable components that make the framework flexible, data-driven and maintainable.

1 Page Object Model (POM)



- One class = One Page
- Keeps locators & actions together
- Easy to maintain and reuse

LoginPage.ts

```ts
import { Page, Locator } from '@playwright/test';
export class LoginPage {
  private page: Page;

  constructor(page: Page) {
    this.page = page;
  }

  private username: Locator = this.page.locator('#username');
  private password: Locator = this.page.locator('#password');
  private loginBtn: Locator = this.page.locator('#login');

  async login(user: string, pass: string) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.loginBtn.click();
  }
```

HomePage.ts

```ts
import { Page, Locator } from '@playwright/test';
export class HomePage {
  constructor(private page: Page) {}

  private logoutBtn: Locator = this.page.locator('#logout');

  async logout() {
     await this.logoutBtn.click();
  }
```

2 Config & Configuration

External config makes tests environment independent.

config.yaml

```yaml
env: qa
baseUrl: https://demo.playwright.dev
browser: chromium
headless: true
timeout: 30000
```

ConfigReader.ts

```ts
import fs from 'fs';
import yaml from 'js-yaml';

export const config = yaml.load(
   fs.readFileSync('config.yaml', 'utf8')
) as any;
```

3 Utility Classes

Reusable helper classes to improve productivity and reduce duplication.

WaitUtils.ts

```ts
import { Locator } from '@playwright/test';
export class WaitUtils {
  static async waitForVisible(locator: Locator, timeout = 10000) {
     await locator.waitFor({ state: 'visible', timeout });
  }
}
```

ScreenshotUtils.ts

```ts
import { Page } from '@playwright/test';
export class ScreenshotUtils {
  static async capture(page: Page, name: string) {
     await page.screenshot({ path: `screenshots//${name}.png`, fullPage: true });
  }
}
```

PLAYWRIGHT AUTOMATION FRAMEWORK

UTILITY CLASSES & COMMON HELPERS

Reusable helper classes to improve productivity and reduce code duplication.

1 WaitUtils



Contains common wait methods to make tests stable.

WaitUtils.ts

```ts
import { Locator, Page } from '@playwright/test';

export class WaitUtils {
  static async waitForVisible(locator: Locator, timeout = 10000) {
    await locator.waitFor({ state: 'visible', timeout });
  }

  static async waitForElementToBeHidden(locator: Locator, timeout = 10000) {
    await locator.waitFor({ state: 'hidden', timeout });
  }
```

2 ScreenshotUtils



Helps to capture screenshots on failure or on demand.

ScreenshotUtils.ts

```ts
import { Page } from '@playwright/test';

export class ScreenshotUtils {
  static async capture(page: Page, name: string) {
    const fileName = `${name}-${Date.now()}.png`;
    await page.screenshot({
      path: `./reports/screenshots/${fileName}`,
      fullPage: true
    });
  }
```

3 DateUtils



Reusable date operations and formatters.

DateUtils.ts

```ts
export class DateUtils {
  static getCurrentDate(format: string = 'yyyy-MM-dd'): string {
    const today = new Date();
    return today.toISOString().split('T')[0];
  }
}
```

4 FileUtils



Read / Write files and handle test data files easily.

FileUtils.ts

```ts
import * as fs from 'fs';

export class FileUtils {
  static readFile(filePath: string): string {
    return fs.readFileSync(filePath, 'utf8');
  }

  static writeFile(filePath: string, data: string) {
    fs.writeFileSync(filePath, data, 'utf8');
  }
}
```

5 LoggerUtils



Simple logger to print logs in a structured way.

LoggerUtils.ts

```ts
export class LoggerUtils {
  static info(message: string) {
    console.log(`\x1b[32m[INFO]\x1b[0m ${message}`);
  }

  static error(message: string) {
    console.error(`\x1b[31m[ERROR]\x1b[0m ${message}`);
  }
}
```

PLAYWRIGHT AUTOMATION FRAMEWORK

TEST EXECUTION & FRAMEWORK FLOW

Brings everything together and executes tests in a clean & maintainable way.

1 Test Example



Actual test using Page Objects and Utilities.

login.spec.ts

```ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { WaitUtils } from '../utils/WaitUtils';

test('User should login successfully', async ({ page }) => {
  const loginPage = new LoginPage(page);

  await page.goto('/login');
  await WaitUtils.waitForVisible(loginPage.username);
  await loginPage.login('standard_user', 'secret_sauce');
  await expect(page.locator('.title')).toHaveText('Products');
});


```

2 test.config.ts



Playwright configuration for projects, retries, reporter & more.

test.config.ts

```ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 1,
  use: { baseURL: process.env.BASE_URL || 'https://demo.com' },
  projects: [ { name: 'chromium', use: { ...devices['Desktop Chrome'] } } ],
  reporter: [['html'], ['list']],
});
```

3 package.json



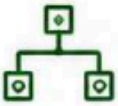
Scripts to run tests, generate report and more.

package.json

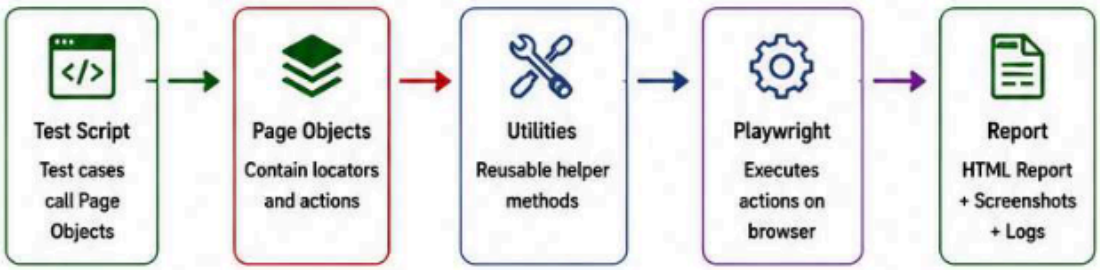
```json
{
  "scripts": {
    "test": "playwright test",
    "test:ui": "playwright test --ui",
    "report":  "playwright show-report"
  }
}

```

4 Framework Flow



How all components work together during execution.



What We Get?

- ✓ Modular & maintainable code
- ✓ Reusability with Page Objects & Utilities
- ✓ Easy configuration & execution
- ✓ Clear reports and logs for debugging